

RAPPORT DE RÉFLEXION

Git, Intégration Continue et Assurance Qualité

Yaovi Urbain · Juin 2026

1. Défis rencontrés

L'apprentissage de Git, de l'intégration continue et des outils d'assurance qualité a présenté plusieurs défis concrets. Le premier défi a été la compréhension du modèle de branches Git : distinguer la branche principale (main) des branches de fonctionnalités, et comprendre pourquoi cette séparation est essentielle pour éviter d'introduire du code instable en production. La résolution de conflits de fusion, bien que non survenue dans ce projet, reste un concept qui nécessite une pratique régulière pour être maîtrisé.

La configuration de GitHub Actions a représenté un autre défi. La syntaxe YAML est sensible à l'indentation, et un simple espace mal placé peut empêcher le pipeline de démarrer. La localisation du fichier ci.yml dans le dossier .github/workflows/ avec la bonne structure a nécessité plusieurs tentatives avant de fonctionner correctement sur GitHub.

Concernant ESLint, la configuration initiale a posé des questions sur le choix de l'environnement (browser vs node). Une mauvaise sélection a nécessité une correction manuelle du fichier de configuration, ce qui a permis de mieux comprendre le rôle de chaque paramètre dans la configuration d'un linter.

2. Apports du pipeline CI

Le pipeline d'intégration continue a transformé fondamentalement la façon d'aborder le développement. Avant sa mise en place, les tests devaient être lancés manuellement — une étape facilement oubliée sous la pression du temps. Avec GitHub Actions, chaque push déclenche automatiquement l'exécution des 15 tests Jest et l'analyse ESLint, en moins de 30 secondes.

Cela apporte une sécurité immédiate : si une modification casse un test existant, le pipeline échoue et une notification est envoyée avant même que le code ne soit fusionné dans la branche principale. Cette détection précoce des régressions est l'un

des apports les plus précieux du CI dans un environnement professionnel, où le coût d'un bug découvert en production est infiniment supérieur à celui détecté en développement.

Le pipeline a également rationalisé le processus de revue de code : lors de l'ouverture de la Pull Request, le statut du pipeline CI est visible directement sur GitHub, donnant aux relecteurs une validation automatique avant même la revue humaine.

3. Impact sur la collaboration et la qualité

Les pratiques de contrôle de version et d'assurance qualité ont amélioré la collaboration de plusieurs façons. Le workflow Git basé sur les branches isole les développements en cours, permettant à plusieurs développeurs de travailler simultanément sans se bloquer mutuellement. Les Pull Requests créent un espace de discussion structuré autour des modifications de code, favorisant le partage de connaissances et la détection collective des erreurs.

ESLint a imposé un style de code cohérent sur l'ensemble du projet. Sans linter, chaque développeur aurait pu utiliser des conventions différentes (guillemets simples ou doubles, variables non utilisées, etc.), rendant le code difficile à lire et à maintenir. Grâce à l'analyse statique, le code soumis respecte les mêmes standards, quelle que soit la personne qui l'a écrit.

En définitive, ces pratiques transforment le développement logiciel d'une activité individuelle et informelle en un processus collaboratif, traçable et fiable, conforme aux standards de l'industrie du génie logiciel.